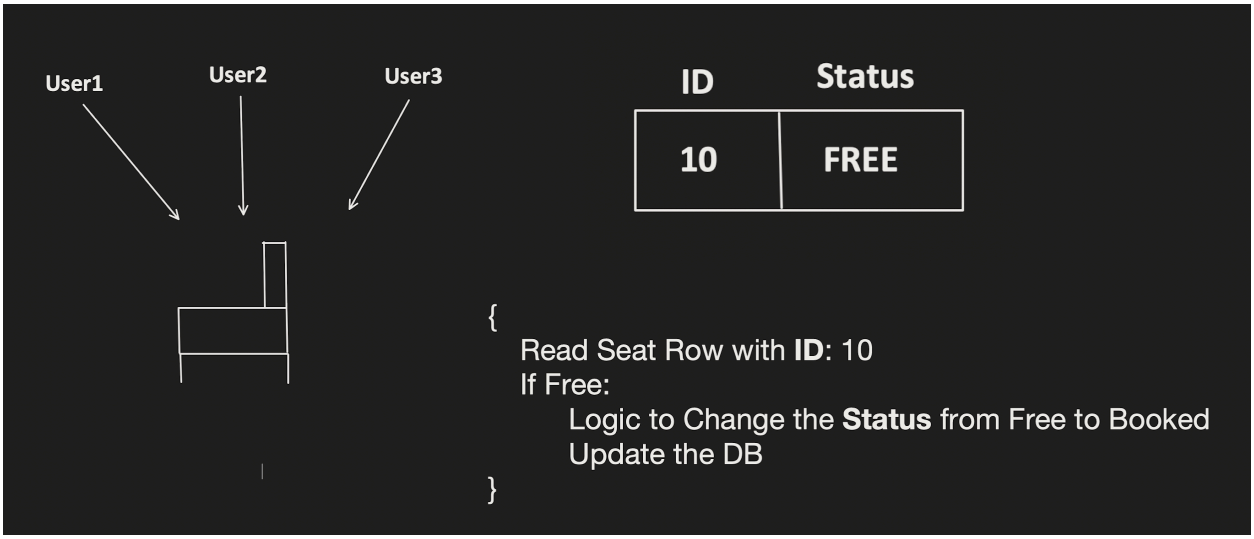
Sign Up

LLD: Concurrency Control

"Concept && Coding" YT Video Notes

Scenario: Many concurrent request tries to book same Movie theatre Seat



1. Using "SYNCHRONIZED" for the Critical Section:

```
synchronized ()
{
    Read Seat Row with ID: 10
    If Free:
        Logic to Change the Status from Free to Booked;
        Update the DB;
}
```

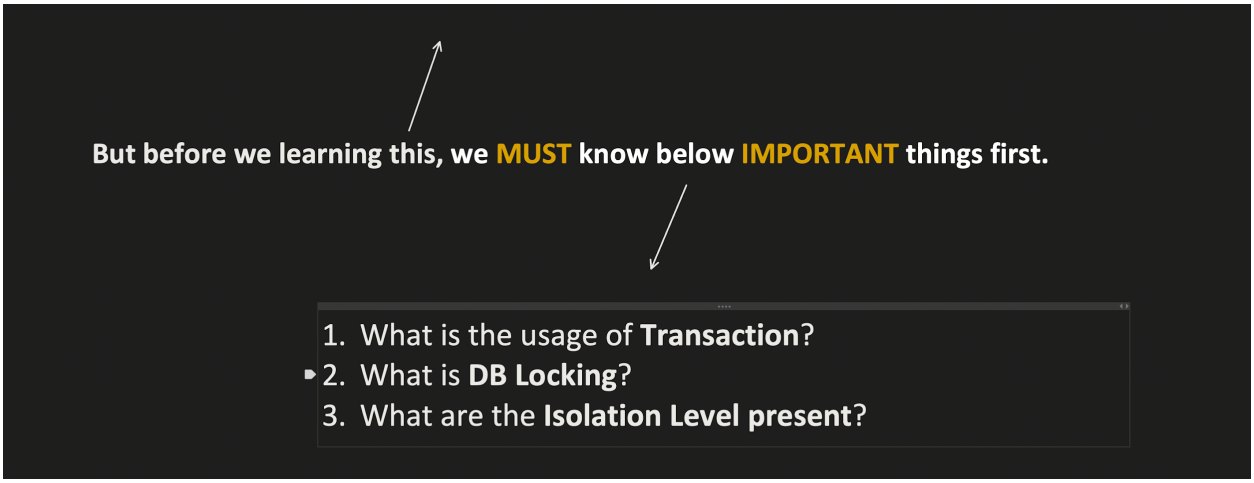
ID	Status
10	FREE

Will this solution works for Distributed System?

2. Using Distributed Concurrency Control

Optimistic Concurrency Control (OCC)

Pessimistic Concurrency Control (PCC)


But before we learning this, we MUST know below IMPORTANT things first.
1. What is the usage of Transaction?
2. What is DB Locking?
3. What are the Isolation Level present?

1. What is the usage of **Transaction**?

Transaction helps to achieve **INTEGRITY.** Means it help us to avoid **INCONSISTENCY** in our database.

For example:

Debit the Money from A and Credit the money to B

BEGIN_TRANSACTION:

- Debit the Money from A
- Credit the Money to B
If all success:
COMMIT;
else:
ROLLBACK;

END_TRANSACTION;

2. What is **DB Locking**?

DB Locking, help us to make sure that no other transaction update the locked rows.

Lock Type	Another Shared Lock	Another Exclusive Lock
Have Shared Lock	Yes	NO
Have Exclusive Lock	NO	NO

3. What are the **Isolation Level present**?

Isolation Level	*Dirty Read Possible*	*Non-Repeatable Read Possible*	*Phantom Read Possible*
Read Uncommitted	Yes	Yes	Yes
Read Committed	No	Yes	Yes
Repeatable Read	No	No	Yes
Serializable	No	No	No

↑ ***Consistency High***

Consistency Low

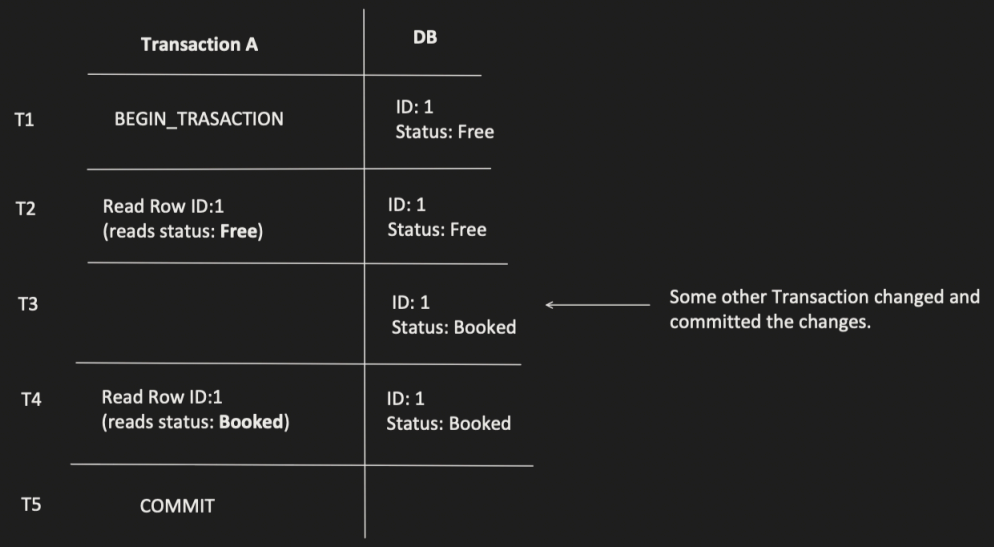
Dirty Read :

If Transaction A is reading the data which is writing by Transaction B and not yet even committed.
If Transaction B does the rollback, then whatever data read by Transaction A is know dirty read.

	Transaction A	Transaction B	DB
T1	BEGIN_TRASACTION	BEGIN_TRASACTION	ID: 1 Status: Free
T2	Update Row ID:1, Status: Booked		ID: 1 Status: Booked (Not Committed by Transaction B Yet)
T3		Read Row ID:1 (Got status as Booked)	ID: 1 Status: Booked (Not Committed by Transaction B Yet)
T4	Some Issue faced here	Some Computation	ID: 1 Status: Booked (Not Committed by Transaction B Yet)
T5	Rollback	Commit	ID: 1 Status: Free

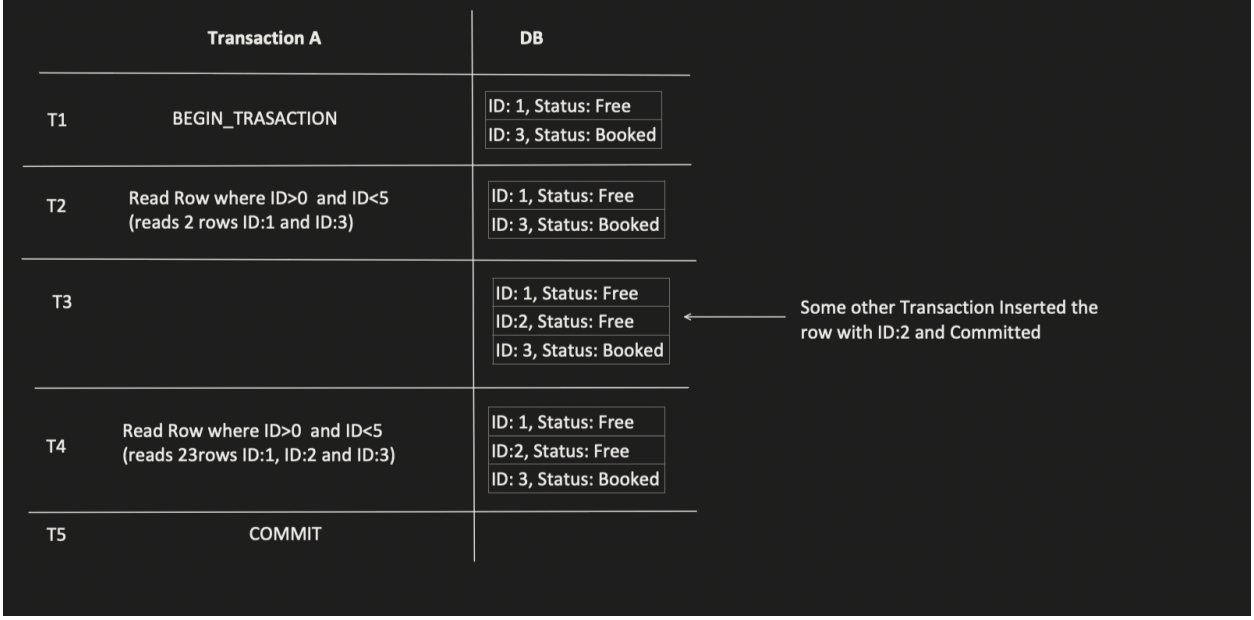
Non-Repeatable Read :

If suppose **Transaction A**, reads the same row several times and there is a chance that it reads different value.



Phantom Read :

If suppose **Transaction A**, executes same query several times and there is a chance that the rows retuned are different.



Lets come back to the Table Again:

Isolation Level	*Dirty Read Possible*	*Non-Repeatable Read Possible*	*Phantom Read Possible*
Read Uncommitted	Yes	Yes	Yes
Read Committed	No	Yes	Yes
Repeatable Read	No	No	Yes
Serializable	No	No	No

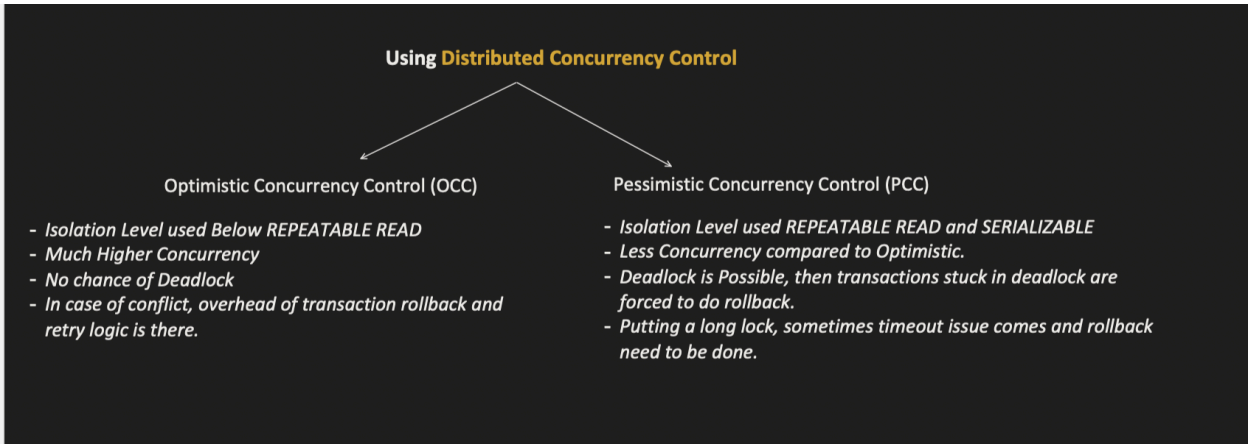
Consistency High

Consistency Low

ISOLATION LEVEL	Locking Strategy
Read Uncommitted	**Read** : No Lock acquired **Write**: No Lock acquired
Read Committed	**Read**: Shared Lock acquired and Released as soon as Read is done **Write**: Exclusive Lock acquired and keep till the end of the transaction
Repeatable Read	**Read**: Shared Lock acquired and Released only at the end of the Transaction **Write**: Exclusive Lock acquired and Released only at the end of the Transaction
Serializable	Same as Repeatable Read Locking Strategy + apply Range Lock and lock is release only at the end of the Transaction.

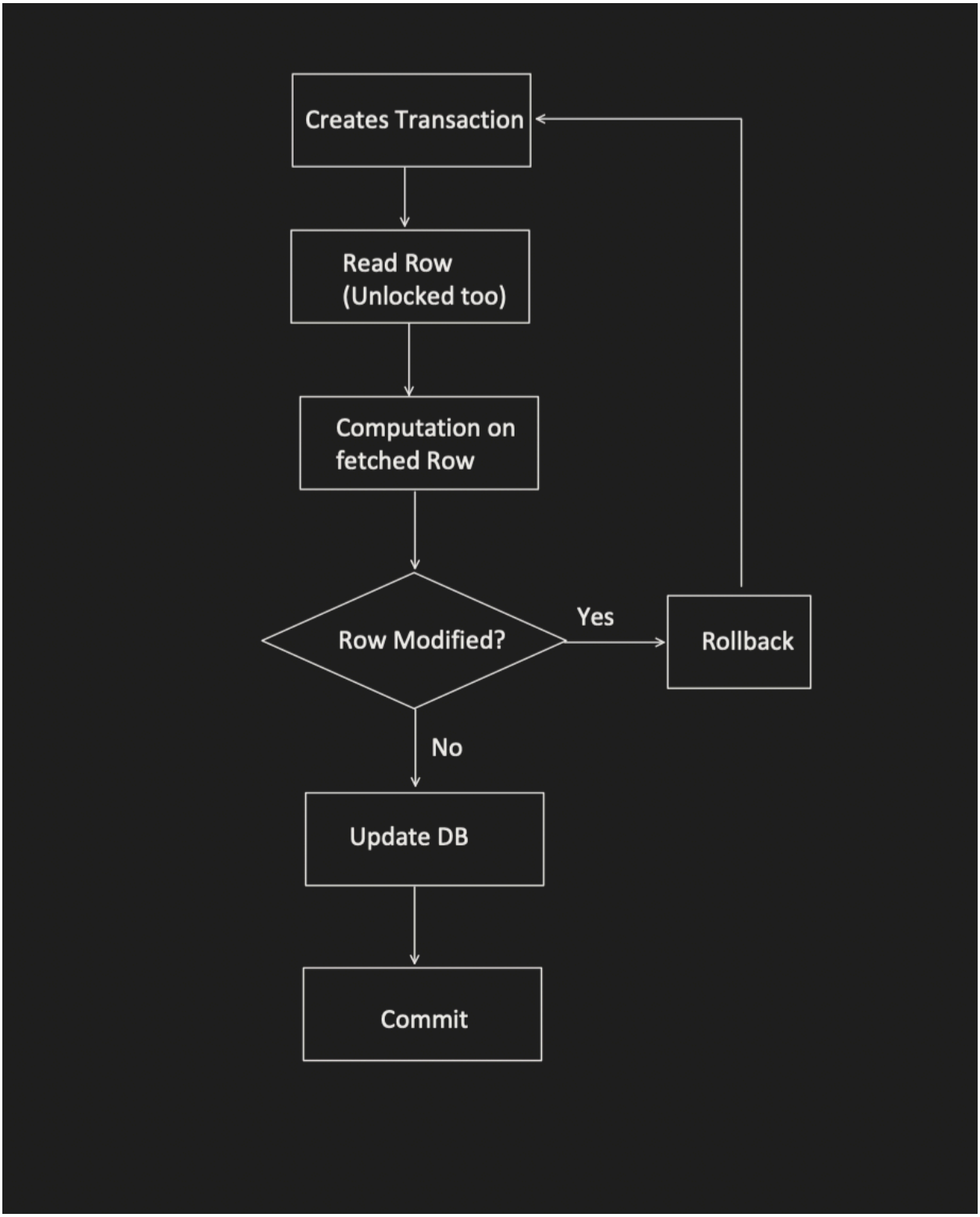
```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

BEGIN_TRANSACTION;

SELECT Query;
Update Query;
If Success:
    COMMIT TRANSACTION;
else:
    ROLLBACK TRANSACTION;

END_TRANSACTION
```

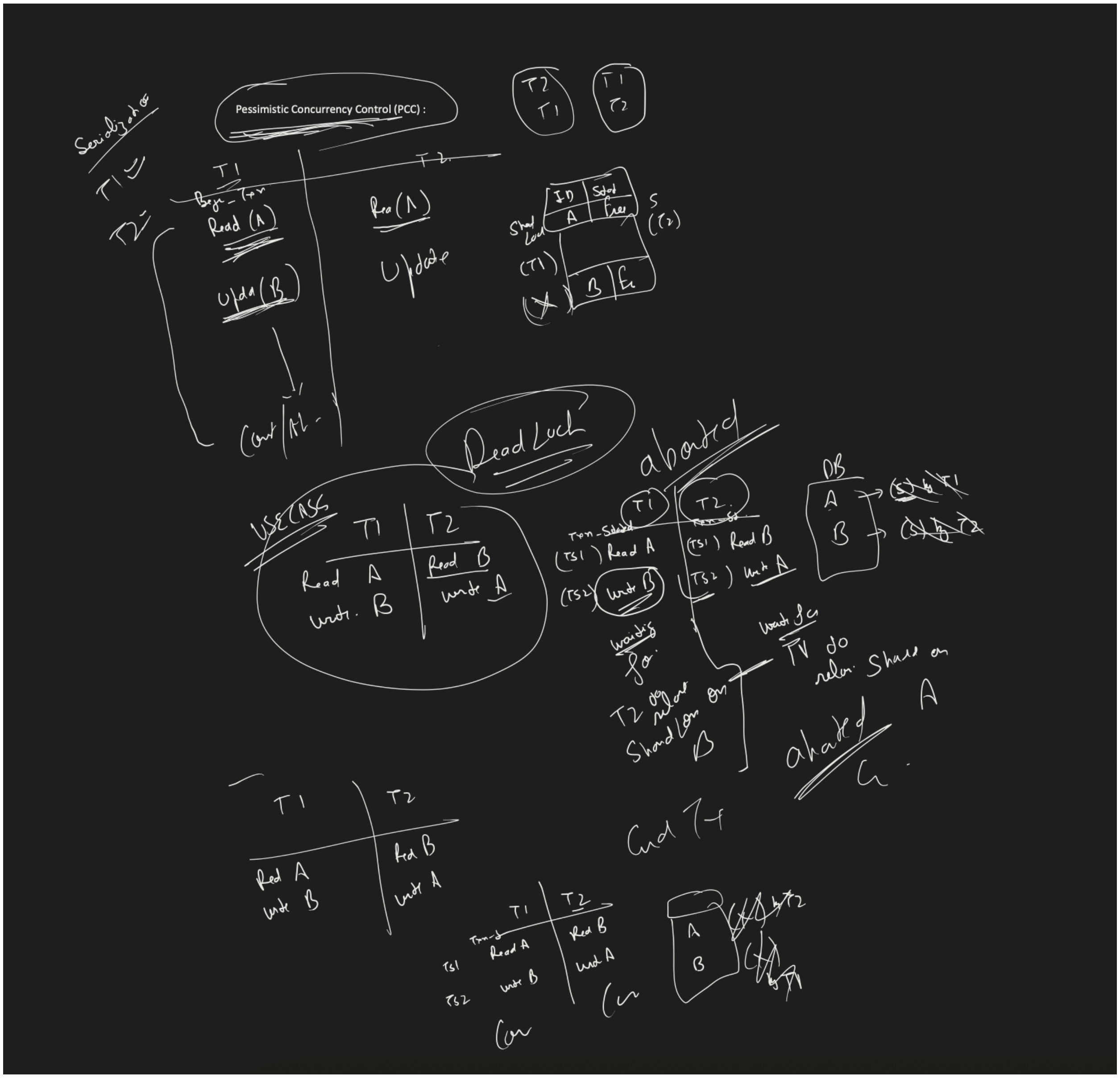
Lets come back to the main Solution of the problem "How to resolve concurrency issue"


Using Distributed Concurrency Control
Optimistic Concurrency Control (OCC)
- Isolation Level used Below REPEATABLE READ
- Much Higher Concurrency
- No chance of Deadlock
- In case of conflict, overhead of transaction rollback and retry logic is there.
Pessimistic Concurrency Control (PCC)
- Isolation Level used REPEATABLE READ and SERIALIZABLE
- Less Concurrency compared to Optimistic.
- Deadlock is Possible, then transactions stuck in deadlock are forced to do rollback.
- Putting a long lock, sometimes timeout issue comes and rollback need to be done.

Optimistic Concurrency Control (OCC) :

	Transaction A	Transaction B	DB
T1	BEGIN_TRASACTION	BEGIN_TRASACTION	ID: 1 Status: Free Version: 1
T2	Read Row ID:1 (Row Version is 1)	Read Row ID:1 (Row Version is 1)	ID: 1 Status: Free Version: 1
T3	Select for Update (Version Validation happens)		ID: 1 Status: Free (Exclusive Lock by Txn A) Version: 1
T4	Update Row ID:1, Status: Booked, Version:2		ID: 1 Status: Booked (Exclusive Lock by Txn A) Version:2
T5	COMMIT		ID: 1 Status: Booked Version:2
T6		Select for Update (Version Validation happens)	ID: 1 Status: Booked (Exclusive Lock by Txn B) Version:2
T7		ROLLBACK	ID: 1 Status: Booked Version:2


Creates Transaction
Read Row
(Unlocked too)
Computation on
fetched Row
Row Modified?
Yes
Rollback
No
Update DB
Commit


Pessimistic Concurrency Control (PCC) :

Report Abuse